

Documento de Requerimientos y Arquitectura Base

v2 — Afinado

Proyecto: Agente Autónomo de Grid Trading (Binance Futures Testnet)

Fase: 0 — Arquitectura e Infraestructura Base

Fecha de revisión: 28 de junio de 2026

Este documento afina la propuesta original de Fase 0. Incluye un hallazgo crítico de validación contra la API real de Binance, una matriz de requerimientos ampliada (R-01 a R-07) y el script del microservicio corregido.

0. Hallazgo Crítico de Validación (leer antes de continuar)

Antes de afinar la redacción del documento, se validó cada endpoint de Binance contra la documentación oficial (developers.binance.com) y el foro de desarrolladores. Se encontró un problema que afecta directamente a R-03 y al script de la sección 4 de la propuesta original:

Binance no expone un endpoint público para crear ni cerrar un “Grid Strategy” nativo en Futures.

El feature de Grid Trading visible en la app/web de Binance es exclusivamente de interfaz (UI); no existe POST /fapi/v1/strategy/grid ni DELETE equivalente en la API pública de Futures.

Confirmado en el foro oficial de desarrolladores de Binance: “Currently you can't start or stop a strategy directly via the Futures API” (dev.binance.vision, hilo “Binance Strategy Futures Grid API”).

El stream STRATEGY_UPDATE / GRID_UPDATE del User Data Stream solo informa sobre grids creados desde la UI; no sirve para crearlos por API.

Impacto: si el equipo construye Fase 0 asumiendo ese endpoint, el primer intento de lanzar un grid devolverá 404 y todo el trabajo de orquestación en n8n quedará bloqueado.

Corrección adoptada en este documento y en el script adjunto:

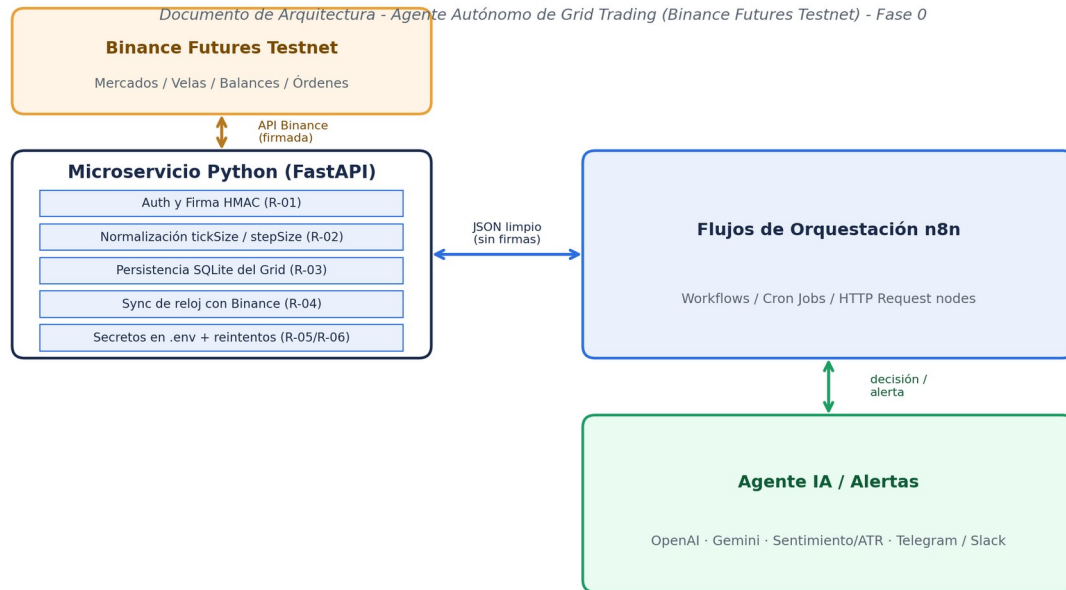
El microservicio Python implementa su propio motor de grid sobre los endpoints documentados y estables de Binance: calcula los niveles de precio, coloca órdenes LIMIT individuales con /fapi/v1/order y /fapi/v1/batchOrders, y persiste el estado (grid_id propio + order_ids de Binance) en SQLite para poder cancelarlas todas en un apagado. Esto se refleja en R-03 y en app_architecture_core.py.

Nota: el cálculo de cuánta cantidad colocar por nivel (sizing) es un placeholder de Fase 0 y debe validarse con el responsable de estrategia antes de operar, incluso en testnet.

1. Arquitectura de Integración (revisada)

Se mantiene el enfoque híbrido de la propuesta original, con la corrección de la sección 0: el Ingeniero de n8n se enfoca en orquestación (workflows, cron jobs, alertas a Slack/Telegram y la capa de IA para leer sentimiento/ATR); el Ingeniero de Python construye el

microservicio de control financiero (FastAPI) que abstrae firmas, normalización de decimales y, ahora, el motor de grid propio.



n8n nunca ve una API key ni calcula una firma: solo consume JSON limpio desde el microservicio Python, que es el único componente con credenciales.

2. Matriz de Requerimientos Técnicos (Checklist de Arquitectura)

Se conservan R-01 a R-04 de la propuesta original (con criterios de aceptación más precisos) y se agregan tres requerimientos nuevos —R-05, R-06 y R-07— que cubren huecos de seguridad y resiliencia que la versión original no contemplaba explícitamente.

Req.	Requerimiento	Estado (Fase 0)
R-01	Gestión Core de Autenticación y Firma (exclusivo backend) Descripción: Centralizar la firma HMAC-SHA256 y la inyección de timestamp y recvWindow. Criterio de aceptación: n8n no calcula firmas en nodos de JavaScript; envía un POST simple con los parámetros del Grid. El backend firma y despacha hacia testnet.binancefuture.com a través de un único punto de salida (binance_request), incluyendo recvWindow explícito (≤5000ms, recomendado por Binance).	Implementado
R-02	Módulo de Normalización Matemática (filtros de mercado) Descripción: Interceptación obligatoria de /fapi/v1/exchangeInfo, con cache de 1 hora. Criterio de aceptación: Función estricta que trunca (no redondea) precios/cantidades al tickSize/stepSize usando Decimal, evitando errores de punto flotante. Si el notional no supera minNotional (campo “notional” en Futures), debe abortar con 422 antes de tocar Binance.	Implementado
R-03	Orquestación de Estados del Grid (motor propio) Descripción: Dado que Binance no expone creación/cierre de Grid	Implementado

	vía API pública (sección 0), el sistema calcula sus propios niveles y coloca órdenes LIMIT individuales. Criterio de aceptación: Al lanzar un grid se almacena en SQLite un grid_id propio (UUID) junto con el símbolo, el rango de precios y los order_ids de Binance asociados. DELETE /api/v1/strategy/grid/{grid_id} cancela exactamente esas órdenes (y opcionalmente cierra la posición) ante un Stop Loss.	
R-04	Sincronización de Tiempo Avanzada Descripción: Mitigación del desfase del reloj local frente a Binance. Criterio de aceptación: El backend consulta /fapi/v1/time al iniciar y cada hora (configurable), calcula el offset corrigiendo la latencia de ida y vuelta, y lo aplica a cada timestamp firmado.	Implementado
R-05	Gestión de Secretos y Entorno Descripción: Las credenciales de Binance no deben vivir en el código fuente. Criterio de aceptación: API_KEY y SECRET_KEY se cargan desde variables de entorno (.env vía python-dotenv); el servicio falla rápido al iniciar si faltan; .env está en .gitignore; los logs nunca imprimen el secreto ni la firma completa.	Implementado
R-06	Resiliencia y Manejo de Rate Limits Descripción: Las llamadas a Binance deben tolerar fallas transitorias y respetar los límites de peso del exchange. Criterio de aceptación: Toda llamada saliente usa timeout explícito y reintentos con backoff exponencial, y maneja explícitamente HTTP 429/418 respetando el header Retry-After cuando está presente.	Implementado
R-07	Idempotencia y Healthcheck Descripción: n8n debe poder verificar que el servicio está disponible antes de disparar un workflow, y evitar lanzar grids duplicados por reintentos de n8n. Criterio de aceptación: GET /api/v1/health responde 200 con el estado del offset de reloj. La validación de “no duplicar grid activo por símbolo” queda pendiente para Fase 1 (ver Próximos Pasos).	Parcial

3. Cambios Respecto a la Versión Original

Resumen de lo que se corrigió o se agregó en esta versión, para que el equipo entienda qué cambió y por qué:

- Corrección crítica: se eliminó la llamada a POST/DELETE `/fapi/v1/strategy/grid` (no existe en la API pública de Futures); se reemplazó por un motor de grid propio sobre `/fapi/v1/order` y `/fapi/v1/batchOrders`.
- Seguridad: las API keys dejaron de estar escritas en el código; ahora se cargan desde `.env` (python-dotenv), con fail-fast si faltan.
- Matemática: el truncado de precios/cantidades pasó de `round()` —que puede redondear hacia arriba y violar el `tickSize/stepSize`— a un truncado estricto con `Decimal` y `ROUND_DOWN`.
- Persistencia: se implementó el esquema SQLite real (tabla grids) con los endpoints GET y DELETE que la propuesta original mencionaba como requerimiento pero no incluía en el script.
- Reloj: se implementó la sincronización periódica con `/fapi/v1/time`; la propuesta original solo lo describía como requerimiento, sin código asociado.
- Resiliencia: se agregaron timeouts, reintentos con backoff exponencial y manejo explícito de 429/418; la versión original no manejaba errores de red ni rate limits.
- Rendimiento: obtener `_filtros_mercado` ahora cachea `exchangeInfo` por 1 hora en lugar de pedirlo en cada request.
- Detalle menor: se corrigió el typo “uvicon” → “uvicorn” en el comentario de arranque del script.

4. Archivos Entregados en esta Fase

- `app_architecture_core.py` — microservicio FastAPI corregido (R-01 a R-07).
- `requirements.txt` — dependencias exactas para levantar el servicio.
- `.env.example` — variables de entorno necesarias, sin valores reales.
- `.gitignore` — excluye `.env` y la base SQLite del control de versiones.

El código completo no se reproduce en este documento para mantenerlo legible; vive en los archivos adjuntos, que son la fuente de verdad.

5. Próximos Pasos para el Equipo de Ingeniería

1. Instalar dependencias: crear un entorno virtual y correr `pip install -r requirements.txt`.
2. Configurar credenciales: copiar `.env.example` a `.env` y completar `BINANCE_API_KEY` / `BINANCE_SECRET_KEY` con las claves de Binance Futures Testnet (no las de Spot).
3. Levantar el servicio: `uvicorn app_architecture_core:app --reload` (por defecto en `http://localhost:8000`).
4. Probar antes de conectar n8n: GET `/api/v1/health` y luego GET `/api/v1/balance` desde el navegador, curl o Postman.
5. Validar con el responsable de estrategia la fórmula de sizing por nivel (`capital_por_nivel`) antes de lanzar el primer grid, incluso en testnet.

6. Conectar n8n: nodo HTTP Request GET a `/api/v1/balance`, y POST a `/api/v1/strategy/grid` con el JSON de configuración del Grid.
7. Definir en n8n el flujo de apagado: bajo qué condición (ATR, sentimiento, drawdown) se llama a DELETE `/api/v1/strategy/grid/{grid_id}`.
8. Fase 1 (pendiente): agregar idempotencia real (rechazar un nuevo grid si ya hay uno ACTIVO para el mismo símbolo) y reconciliación de fills vía el User Data Stream de Binance.

6. Notas de Seguridad antes de Pasar a Producción

Este documento cubre Testnet. Antes de mover el agente a mainnet con capital real, se recomienda repetir esta checklist agregando:

- Un límite de pérdida diaria configurable que detenga el agente automáticamente.
- Un kill-switch manual accesible desde Telegram/Slack para apagar todos los grids activos en un solo comando.
- Alertas de margen y de uso de peso de la API (X-MBX-USED-WEIGHT) para anticipar bloqueos temporales.
- Revisión de permisos de la API key de producción: solo Futures Trading habilitado, retirios (withdrawals) deshabilitados.
- Separación estricta de entornos: las claves de testnet y mainnet nunca deben coexistir en el mismo archivo `.env` ni en el mismo proceso en ejecución.